

ESTUDO COMPARATIVO ENTRE A COMPUTAÇÃO SEQUENCIAL E A COMPUTAÇÃO PARALELA APLICADAS AOS ALGORITMOS DE ORDENAÇÃO

Claudineia Moreira de Oliveira
Leandro Estevão de Mello
Lieger Duarte de Oliveira Rosa
Neideson Tavares Viana

Universidade Federal Fluminense
Escola de Engenharia Industrial Metalúrgica de Volta Redonda - EEIMVR
Mestrado em Modelagem Computacional em Ciência e Tecnologia - MCCT

Resumo

O presente relatório abordará a programação de alto desempenho, os algoritmos de ordenação e a comparação destes, usando-se a computação sequencial e a programação paralela. Para isto, criou-se um vetor composto por números inteiros que foram gerados por um processo pseudo-aleatório. Os vetores possuem milhares de elementos, os quais foram divididos em pequenos blocos que comportam quatro ou cinco elementos, com a ordem de criação dos blocos também seguindo um processo pseudo-aleatório. Para fazer a ordenação dos números (elementos do vetor divididos nos blocos), selecionou-se um algoritmo de ordenação para atuar no bloco par (bloco com quatro elementos), outro para atuar no bloco ímpar (blocos com cinco elementos) e outro para atuar no processo de união dos blocos ordenados, mas que ainda caracterizavam um vetor não ordenado. Feita a escolha dos algoritmos de ordenação, construiu-se um algoritmo mais geral - na linguagem C++, mas que empregou muitos elementos característicos da linguagem C - que empregou a computação sequencial e a computação paralela, visando verificar qual delas possuía um desempenho melhor em termos do tempo de execução para a ordenação do vetor na metodologia exposta acima. Acerca da computação paralela, usou-se a paralelização manual - a qual se baseou em suporte nativo para *threads* - e a paralelização automática que fez uso da biblioteca *OpenMP*. Por fim, os resultados foram apresentados, permitindo a comparação de desempenho dos diferentes tipos de computação.

Palavras-chave — Algoritmos de ordenação, computação sequencial, computação paralela, *Threads*, *OpenMP*, C, C++.

1. INTRODUÇÃO

A disciplina de Estrutura de Dados é parte fundamental da formação de um profissional que deseja atuar na área de Ciências da Computação ([1], [2]). Entre os tópicos estudados, encontram-se a programação orientada ao objeto, as listas lineares, as árvores, os grafos, a programação paralela, a programação distribuída e os algoritmos de ordenação.

O presente relatório abordará um tópico muito importante: a diferença de desempenho entre a computação sequencial e a computação paralela. A computação sequencial, como o próprio nome sugere, executa as tarefas de forma sequencial, lembrando a lógica de uma fila indiana, onde várias pessoas entram em um edifício, porém, uma de cada vez. Nesta ilustração, as pessoas seriam as tarefas a serem executadas e a entrada do edifício seria a capacidade de processamento das tarefas, as quais são feitas uma de cada vez. Deve-se salientar que a computação sequencial está baseada na arquitetura de *von Neumann* [3].

A computação paralela surge tanto a nível de *hardware* (sistemas paralelos) quanto a nível de *software* (programas paralelos) [3]. Os sistemas paralelos surgiram em resposta ao problema do aumento da densidade dos transistores nos circuitos integrados que, até certo ponto, aumentou o poder de processamento da Unidade Central de Processamento (em inglês, *Central Processing Unit* ou CPU) composta por um único núcleo (em inglês, *core*) [3]. Porém, à medida que esse processo avançava, aumentou-se o consumo de energia e, consequentemente, a dissipação de parte da mesma na forma de calor, tornando-se inviável para o bom funcionamento dos componentes eletrônicos [3]. Em vez de se construir CPUs mais complexas e rápidas com um único núcleo, decidiu-se criar sistemas com vários núcleos, o que caracteriza os sistemas paralelos [3].

Visando aproveitar as vantagens trazidas pelos sistemas paralelos, desenvolveram-se os programas paralelos [3], os quais podem ser implementados em alto nível - o qual oferece uma maior simplicidade para o usuário, pois ele não tem que se envolver nos detalhes da paralelização - ou em baixo nível - menor simplicidade, mas oferece ao usuário maior controle sobre cada etapa da paralelização. Se os programas paralelos forem executados em uma mesma máquina, pode-se usar os recursos da programação paralela para arquiteturas de memória compartilhada que, ao ser implementados em alto nível, usam as ferramentas da biblioteca *OpenMP*.

Mas se forem implementadas a baixo nível, usam a *Pthreads*. Caso haja a disponibilidade de vários computadores, podem-se usar os recursos da programação paralela para arquiteturas de memória distribuídas que estão na esfera de atuação do MPI (em inglês, *Message Passing Interface*).

Neste relatório, o foco será em comparar o desempenho entre a computação tradicional e a computação paralela em um clássico problema de Ciências da Computação: a ordenação dos elementos de um vetor. Ressalta-se que a computação paralela envolverá um computador com múltiplos núcleos e duas diferentes paralelizações: uma envolvendo os recursos do *OpenMP* e outra envolvendo os recursos da *Pthreads*.

O presente artigo está dividido em cinco tópicos, a saber, Fundamentação Teórica, Metodologia, Resultados e Discussões, Conclusão e Referências. Em Fundamentação Teórica, os algoritmos de ordenação serão brevemente introduzidos e, posteriormente, será feita a apresentação da computação sequencial juntamente com a computação paralela, na qual suas principais características serão destacadas. Em Metodologia, o problema a ser resolvido será apresentado e será mostrada a estratégia adotada para resolvê-lo. Em Resultados e Discussões, o desempenho da computação sequencial e da computação paralela serão mostrados seguido de uma análise detalhada que unirá a teoria apresentada no capítulo anterior com os dados oriundos dos testes computacionais. Por fim, em Conclusão, os principais pontos discutidos no relatório serão destacados e, em Referências, serão apresentados os trabalhos que contribuíram para a confecção deste artigo.

2. FUNDAMENTAÇÃO TEÓRICA

Os algoritmos de ordenação constituem uma das áreas mais fundamentais da ciência da computação, sendo estudados há décadas tanto por sua aplicabilidade prática quanto por sua relevância teórica na análise de complexidade computacional. Segundo Cormen et al. [2], ordenar um conjunto de valores significa reorganizá-los em uma sequência específica, geralmente crescente ou decrescente, operação que serve de base para inúmeros outros algoritmos e sistemas computacionais.

A escolha do algoritmo de ordenação mais adequado depende de características do problema, como o tamanho do conjunto de dados, o grau de ordenação prévia dos elementos e as restrições de memória disponível. Cormen et al. [2] classificam os algoritmos de ordenação em dois grandes grupos: métodos simples, com complexidade $O(n^2)$, adequados para conjuntos pequenos de dados, e métodos eficientes, com complexidade $O(n \log_2 n)$, indicados para grandes volumes de dados. Em [1], reforça-se que os métodos simples, embora menos eficientes assintoticamente, são preferíveis em contextos onde n é pequeno, pois apresentam menor *overhead* de implementação.

O **Insertion Sort**, ou ordenação por inserção, é um algoritmo que percorre o vetor, inserindo cada elemento na posição correta em relação aos elementos já ordenados. Cormen et al. [2] descrevem este algoritmo como intuitivo e análogo à forma como humanos organizam cartas em um jogo: a cada passo, um novo elemento é retirado e inserido na posição adequada entre os elementos já organizados. Sua análise de complexidade revela comportamento assimétrico: no melhor caso, quando o vetor já se encontra ordenado, o algoritmo executa em tempo $\Omega(n)$, realizando apenas comparações sem trocas. No caso médio e no pior caso, quando os elementos estão em ordem inversa, a complexidade é $O(n^2)$ tanto em comparações quanto em movimentações [2]. Essa característica torna o *Insertion Sort* particularmente eficiente para vetores pequenos ou quase ordenados, sendo amplamente utilizado como subalgoritmo em implementações híbridas de ordenação. O funcionamento deste algoritmo é mostrado na Figura 1.

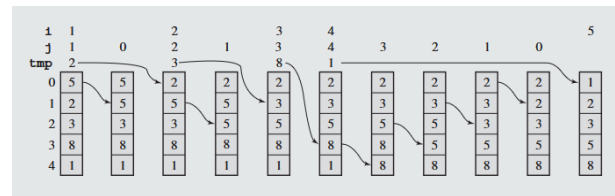


Fig. 1. Aplicação do algoritmo *Insertion Sort* em um vetor de cinco elementos. Note que um elemento é selecionado e, posteriormente, é inserido na posição correta no conjunto dos elementos ordenados que, neste caso, se encontram na parte superior do vetor (elementos de menores índices). Salienta-se que i e j correspondem, respectivamente, ao laço de repetição externo e interno do algoritmo. (Fonte: [1]).

O *Bubble Sort* é um dos algoritmos de ordenação mais simples e didáticos, baseado em comparações adjacentes e trocas sucessivas. A cada passagem pelo vetor, o maior elemento não ordenado é "borbulhado" até sua posição final. Cormen et al. [2] apresentam o *Bubble Sort* como exemplo clássico de análise de invariantes de laço, demonstrando formalmente sua correção. Sua complexidade é $O(n^2)$ tanto no caso médio quanto no pior caso, e $\Omega(n^2)$ no melhor caso quando se implementa uma verificação de ausência de trocas [1]. Observa-se que, apesar de sua simplicidade, o *Bubble Sort* tende a ser mais lento que o *Insertion Sort* e o *Selection Sort* na prática, devido ao maior número de trocas realizadas. O funcionamento deste algoritmo é mostrado na Figura 2.

O *Selection Sort* é também um algoritmo de fácil construção e sua lógica de funcionamento é bem intuitiva e se assemelha ao modo como uma pessoa ordenaria um conjunto composto de números inteiros. Se o objetivo é organizar os elementos deste conjunto em ordem crescente, o primeiro passo que a pessoa executaria seria procurar o menor número e colocá-lo na primeira posição do conjunto. Posteriormente, o segundo

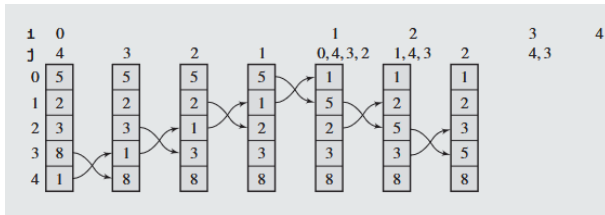


Fig. 2. Aplicação do algoritmo *Bubble Sort* em um vetor de cinco elementos. Note que os maiores números ocuparão os maiores índices do vetor enquanto os menores números ocuparão os menores índices, com essa configuração se concretizando pela comparação dois a dois entre os elementos. Salienta-se que i e j correspondem, respectivamente, ao laço de repetição externo e interno do algoritmo. (Fonte: [1]).

menor número seria procurado e colocado na segunda posição do conjunto. O modo de operação descrito é a maneira pela qual o *Selection Sort* ordena os elementos de um vetor. Ele busca pelo elemento crítico (o menor elemento no caso da organização em ordem crescente) entre todos os elementos e, ao encontrá-lo, ele o posiciona na primeira posição do vetor, conforme mostra a figura 3.

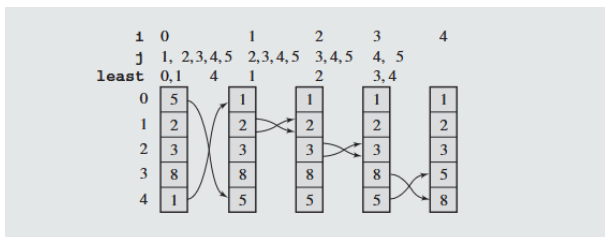


Fig. 3. Aplicação do algoritmo *Bubble Sort* em um vetor de cinco elementos. Note que os maiores números ocuparão os maiores índices do vetor, enquanto os menores números ocuparão os menores índices, com essa configuração se concretizando pela comparação dois a dois entre os elementos. Salienta-se que i e j correspondem, respectivamente, ao laço de repetição externo e interno do algoritmo. (Fonte: [1]).

O *Merge Sort* é um algoritmo de ordenação baseado no paradigma de divisão e conquista. Cormen et al. [2] apresentam sua estrutura em três etapas: dividir o vetor em duas metades, ordenar recursivamente cada metade e, por fim, intercalar (*merge*) as duas metades ordenadas em um único vetor ordenado. Sua complexidade é $O(n \log_2 n)$ em todos os casos — melhor, médio e pior — o que o torna uma das opções mais robustas para grandes conjuntos de dados. Além disso, é um algoritmo estável, preservando a ordem relativa de elementos iguais. Sua principal desvantagem é o uso de memória auxiliar para o processo de intercalação [2]. Uma propriedade especialmente relevante para este trabalho é que o *Merge Sort* é naturalmente eficiente para intercalar sequências já ordenadas. Kittitornkun e Rattanatrakun [4] exploram justa-

mente essa característica em algoritmos paralelos modernos, demonstrando que a etapa de *merge* é o gargalo natural para paralelização. A figura 4 mostra o funcionamento do *Merge Sort*.

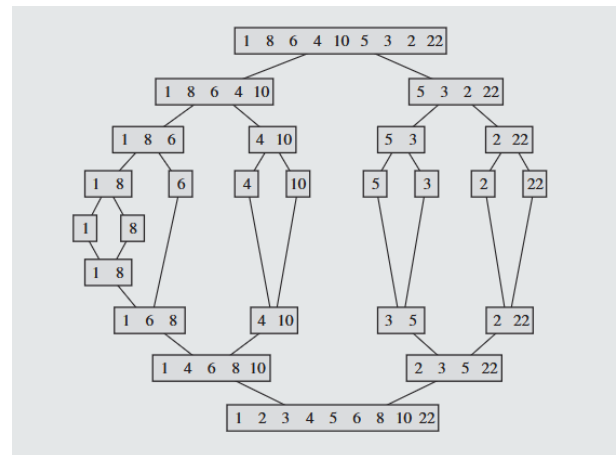


Fig. 4. Aplicação do algoritmo *Merge Sort* em um vetor de nove elementos. Perceba que o algoritmo divide o vetor em subconjuntos, os quais vão ficando cada vez menores (poucos elementos), visando facilitar o processo de comparação. Após os subconjuntos adquirirem um tamanho mínimo, a comparação é realizada entre seus elementos. Posteriormente, os subconjuntos ordenados vão se juntando em subconjuntos maiores e seus elementos são novamente ordenados (veja que a ordenação vai ficando mais fácil uma vez que os subconjuntos originais já foram ordenados). Esse processo se repete até que o vetor seja novamente obtido, porém, desta vez, todos os elementos já foram ordenados. (Fonte: [1]).

Assim como foi visto no *Merge Sort*, o *Quick Sort* utiliza a estratégia "dividir para conquistar" como sua lógica de operação. Porém, diferente do *Merge Sort* que divide o vetor ao meio para formar os subconjuntos, o *Quick Sort* realiza essa formação de subconjuntos por meio de um pivô, tal que um subconjunto será composto por elementos menores que o pivô e o outro subconjunto será composto por elementos maiores que o pivô [1]. Sua complexidade de melhor caso e sua complexidade de pior caso são, respectivamente, $\Omega(n \log_2 n)$ e $O(n^2)$.

Para se resolver problemas de ordenação em vetores com milhares de elementos, pode-se confeccionar um algoritmo que faz uso da computação sequencial. Nela, as tarefas são processadas computacionalmente uma a uma, lembrando o funcionamento de uma fila indiana. Outra característica deste tipo de computação é o uso de um único núcleo para a resolução de problemas computacionais ([3], [5]).

Mesmo com as características apresentadas acima, a computação sequencial pode dar a impressão de executar múltiplas tarefas simultaneamente (conhecida em inglês como *multitasking*) [6], como, por exemplo, quando um

usuário usa o computador para ouvir música e, ao mesmo tempo, consegue digitar um documento em um editor de texto. No entanto, o que acontece por trás desse processo é uma orquestração eficiente do tempo de processamento gerenciado pelo sistema operacional. Como a percepção humana é incapaz de notar variações temporais na ordem de milissegundos, o sistema operacional divide o tempo real em frações minúsculas, de modo que os diferentes programas são executados em sequência nestes pequenos intervalos de tempo [6]. A execução de partes de diferentes programas em frações de tempo minúsculas provoca a sensação de que eles estão sendo executados ao mesmo tempo.

Ainda no contexto da computação sequencial, uma ideia contemporânea às múltiplas tarefas foi a segmentação de instrução que é conhecida em inglês como *pipelining*. Esta técnica se baseia na transformação de Unidade Central de Processamento em uma espécie de linha de montagem dividida em um conjunto de estágios sequenciais e especializados. Para que isso seja possível, cada instrução de um programa é decomposta em partes menores, conforme ilustrado na figura 5.

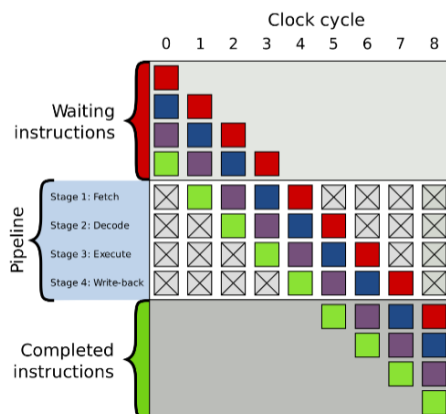


Fig. 5. Exemplo do funcionamento da segmentação de instrução. Note que, inicialmente ($clock = 0$), quatro instruções aguardam para serem processadas (em inglês, *waiting instructions*), com cada instrução sendo representada por um bloco colorido. No primeiro ciclo do relógio ($clock = 1$), a primeira instrução da fila (bloco de cor verde) é colocada no processador e sua primeira parte é processada. No segundo ciclo do relógio ($clock = 2$), a primeira instrução é passada para o segundo estágio, onde sua segunda parte é processada. Simultaneamente, a segunda instrução da fila (bloco de cor roxa) é passada ao primeiro estágio do processador. Perceba que múltiplas instruções são processadas no processador (em inglês, *completed instructions*), porém, cada uma em seu próprio estágio, lembrando a lógica de funcionamento de uma linha de montagem industrial. (Fonte: [6]).

Outra maneira de se resolver o problema de ordenação de um vetor com milhares de elementos é usando a computação

paralela. Ela consiste na execução simultânea de múltiplas tarefas computacionais, com o objetivo de reduzir o tempo total de processamento. Pacheco [3] define paralelismo como a decomposição de um problema em subproblemas independentes, que podem ser resolvidos concorrentemente por diferentes unidades de processamento. Em arquiteturas de memória compartilhada, múltiplos núcleos ou processadores acessam um mesmo espaço de endereçamento, o que simplifica a comunicação entre tarefas, mas exige mecanismos de sincronização para evitar condições de corrida. Neste trabalho, o paralelismo é aplicado especificamente na etapa de ordenação dos blocos, que constitui a fase naturalmente paralelizável do algoritmo: cada bloco é independente dos demais, não existindo dependência de dados entre eles. Essa característica é denominada paralelismo de dados por Pacheco [3], e representa o cenário ideal para exploração de múltiplos núcleos (veja a Figura 6).

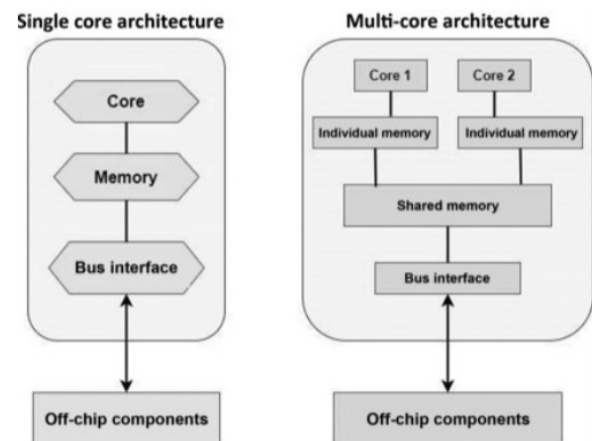


Fig. 6. Arquitetura de um único núcleo (em inglês, *Single core architecture*) e arquitetura de múltiplos núcleos (em inglês, *Multi-core architecture*). Perceba que naquela se vê o núcleo (*core*) ligado à sua memória (*memory*) e esta, por sua vez, está conectada com a interface de barramento (*bus interface*). Já na arquitetura de múltiplos núcleos, observa-se que cada núcleo se conecta com sua memória (*individual memory*) que, por sua vez, liga-se a uma memória compartilhada (*shared memory*). Em ambas as arquiteturas, há conexão com componentes fora do chip (*off-chip components*) que representam qualquer *hardware*, circuito ou memória que opera em um chip separado do processador principal. (Fonte: [6]).

Além do uso de sistemas paralelos, precisa-se desenvolver algoritmos que sejam capazes de aproveitar os múltiplos núcleos do processador. Se estiver disponível o sistema de memória compartilhada (Figura 7 (a)), pode-se usar a paralelização oriunda dos recursos do *OpenMP* ou a paralelização oriunda dos recursos do *Pthreads*, com esta permitindo o controle, por parte do usuário, de todas as etapas da paralelização,

enquanto aquela realiza a geração implícita de *threads*, cabendo ao usuário somente inserir linhas de comando (diretivas de compilação) na região do código que será paralelizada [5]. Assim, enquanto uma oferece maior controle (*Pthreads*) a outra oferece mais facilidade (*OpenMP*) ao usuário, com ambas sendo exploradas neste trabalho.

Por fim, destaca-se que se um sistema de memória distribuída (Figura 7 (b)) reúne diferentes máquinas no esforço de paralelização, fornecendo um cenário ideal para a aplicação da *MPI* (*Message Passing Interface*). Por mais que esse recurso não seja alvo deste relatório, destaca-se seu uso em supercomputadores, *data centers* e em computação em nuvem (*cloud computing*).

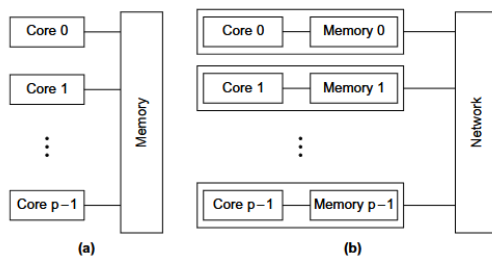


Fig. 7. Em (a), tem-se o sistema de memória compartilhada, o qual fornece o contexto para o uso da paralelização de alto nível (*OpenMP*) e da paralelização de baixo nível (*Pthreads*). Em (b), tem-se o sistema de memória distribuída, o qual fornece o contexto para o uso da paralelização via *MPI*. Em (a), note que os núcleos do computador (*core*) estão conectados à memória (*memory*). Já em (b), diferentes máquinas estão conectadas à rede (*network*) que possibilita o meio pela qual elas irão se comunicar. (Fonte: [3]).

3. METODOLOGIA

Visando comparar o desempenho da computação sequencial e da computação paralela, decidiu-se empregar o problema da ordenação de um vetor com milhares de elementos, sendo eles números inteiros gerados em um processo pseudo-aleatório. Os elementos dos vetores foram divididos em blocos compostos por quatro ou cinco elementos, com a ordem de criação dos blocos sendo feita também por um processo pseudo-aleatório.

Após a criação dos blocos, selecionou-se um algoritmo de ordenação para atuar no bloco com quatro elementos e outro algoritmo para o bloco com cinco elementos. No primeiro caso (bloco com quatro elementos), recomendou-se o uso de um dos três algoritmos: *Selection Sort*, *Merge Sort* e *Insertion Sort*. No segundo caso, recomendou-se o uso de um dos três algoritmos: *Selection Sort*, *Merge Sort* e *Bubble Sort*.

Nos blocos de quatro e cinco elementos, selecionaram-se, respectivamente, o *Insertion Sort* e o *Selection Sort*. Note que

ambos os algoritmos possuem complexidade de melhor e pior caso como sendo, respectivamente, $\Omega(n^2)$ e $O(n^2)$ ([1],[7]), porém, eles foram escolhidos em detrimento do *Merge Sort* que, teoricamente, tem um desempenho superior, possuindo complexidade de melhor e pior caso, respectivamente, como sendo $\Omega(n \log_2 n^2)$ e $O(n \log_2 n^2)$ ([1],[7]). Entretanto, o *Merge Sort* faz uso de chamadas recursivas que são muito eficientes para vetores grandes, mas que, no caso da ordenação de pequenos blocos, possuem um desempenho inferior ao laço de repetição simples (principal recurso do *Insertion Sort* e do *Selection Sort*).

Após a ordenação dos elementos no interior dos blocos, os blocos foram reunidos novamente para formar o vetor. No entanto, mesmo que os elementos no interior dos blocos estejam ordenados, o vetor não está ordenado. No processo de união dos blocos, havia três possibilidades sugeridas para a ordenação dos elementos: *Insertion Sort*, *Merge Sort* e *Quick Sort*. Neste caso, os dois últimos algoritmos possuem uma clara vantagem em relação ao primeiro algoritmo, pois, teoricamente, eles possuem complexidade de melhor como sendo $\Omega(n \log_2 n^2)$. Como o *Quick Sort* possui a complexidade de pior caso como sendo $O(n^2)$ enquanto o *Merge Sort* possui a complexidade de pior caso como sendo $O(n \log_2 n^2)$, optou-se por este algoritmo, devido à sua melhor performance no geral.

Deve-se destacar que o *Merge Sort* é a melhor opção para a etapa de união dos blocos, pois ele atuará em partições já ordenadas, eliminando a necessidade de ele mesmo fazer as partições e otimizando ainda mais seu desempenho. Destaca-se também que o *Bubble Sort* foi excluído entre os algoritmos do segundo caso (ordenar os blocos com cinco elementos), pois seu desempenho é inferior ao dos demais algoritmos. Na Figura 8, tem-se um panorama geral do desempenho dos algoritmos na ordenação de vetores, usando-se a computação sequencial.

Após a seleção dos algoritmos de ordenação para cada etapa, construiu-se um algoritmo na linguagem de programação C++, mas que empregou muitos elementos característicos da linguagem C, visando gerar um código mais simples, mesmo a custo de ser mais extenso.

O algoritmo em C++ resolve o problema da ordenação dos vetores, aplicando a computação sequencial e a computação paralela, com esta sendo implementada por meio de uma paralelização de baixo nível e por meio de uma paralelização de alto nível. Naquela, usando-se uma biblioteca nativa do C++, ajustou-se a paralelização manualmente, com o primeiro passo sendo o uso do comando *hardware_concurrency()* para saber quantas *threads* lógicas o processador suporta. Depois, faz-se o cálculo do número de blocos que cada *thread* precisará ordenar. Além disso, criou-se uma estrutura (*struct*) para reunir todas as informações de que as *threads* precisam, como, por exemplo, os índices que marcam o início e o fim de atuação de cada uma delas. Por fim, dispararam-se as *threads* em paralelo dentro de um laço de repetição (comando

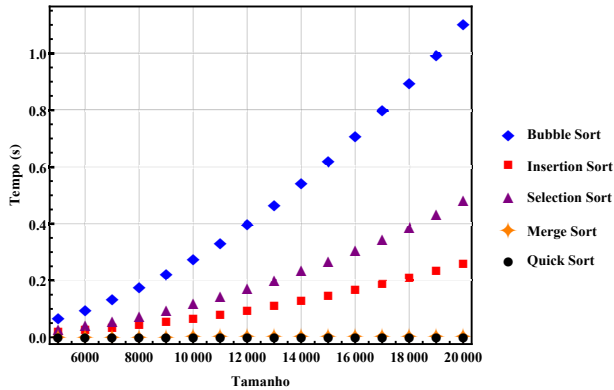


Fig. 8. Desempenho dos principais algoritmos de ordenação para vetores de diferentes tamanhos e usando a computação sequencial. Salienta-se que a metodologia empregada para obtenção dos resultados mostrados na figura difere da metodologia empregada neste trabalho, pois não se usou blocos para dividir os elementos do vetor. Ou seja, os algoritmos de ordenação foram usados diretamente no vetor. Note que, para vetores com poucos elementos, o desempenho dos algoritmos foi muito parecido, porém, à medida que os vetores vão ficando maiores, fica evidente que o *Bubble Sort* tem a pior performance enquanto que *Quick Sort* e o *Merge Sort* tem a melhor performance. (Fonte: Autores).

for), tomando-se o cuidado de sincronizá-las com o programa principal.

Acerca da paralelização de alto nível, usou-se a biblioteca *OpenMP* que realiza a paralelização automática, sem a necessidade do programador gerenciar cada detalhe deste processo, como foi feito no parágrafo anterior. Tudo o que se precisou fazer foi inserir a diretiva de programação `#pragma omp parallel for` logo acima do laço de repetição (comando *for*) que percorre os blocos a serem ordenados.

Por fim, por meio dos recursos da biblioteca *time.h*, mediu-se o tempo gasto na ordenação dos vetores, usando-se a computação sequencial e os dois tipos de computação paralela. Salienta-se que o tempo medido foi o tempo médio de ordenação, visto que, para cada tamanho do vetor, criaram-se trinta vetores diferentes para serem ordenados.

4. RESULTADOS E DISCUSSÕES

O algoritmo foi executado em um *notebook* modelo *HP 240 G4 Notebook PC*, com 6 GB de memória RAM, capacidade de disco de 620,1 GB e processador *Intel Core i3-5005U x 4*. Além disso, usou-se o sistema operacional *open-SUSE Leap 15.6* que é uma distribuição do *Linux*. Consequentemente, o *notebook* é capaz de executar 10^6 clocks/s e possui 2 núcleos físicos e 4 núcleos lógicos, com este últimos sendo usados para o processo de paralelização.

Executando o código confeccionado - o qual pode ser dis-

ponibilizado pelos autores, mediante solicitação por *e-mail* - obtiveram-se os resultados mostrados na Tabela 1.

Esperava-se que os algoritmos paralelizados tivessem um desempenho superior ao algoritmo sequencial, porém, os resultados mostrados na Tabela 1 indicam que o algoritmo sequencial teve um desempenho similar ao algoritmo paralelizado manualmente (*Pthreads*) e um desempenho superior ao algoritmo paralelizado via *OpenMP*.

Observando atentamente o código, vê-se que as etapas comuns dos três algoritmos devem gerar o mesmo tempo de execução, sendo elas a alocação e geração dos dados brutos (preenchimento do vetor com elementos aleatórios), o fatiamento dinâmico (criação dos blocos com quatro ou cinco elementos), cópia e desfatiamento e união final. A pequena diferença de tempo deve vir da ordenação dos blocos, mostrando a superioridade da computação paralela sobre a computação sequencial, no entanto, percebeu-se descuidos na paralelização que fizeram a diferença no resultado final.

Por exemplo, na paralelização via *OpenMP*, aplicou-se a diretiva `#pragma omp` no laço de repetição que percorria os blocos com quatro ou cinco elementos, fazendo com que muito tempo fosse gasto, criando, distribuindo e sincronizando as *threads* a cada iteração, em vez de fazer os cálculos em si. Esse custo de gerenciamento (*overhead*) prejudicou de forma significativa o desempenho do algoritmo.

Outro problema que os autores verificaram foi que não houve uma comparação justa entre as duas formas de paralelização. Enquanto na paralelização manual, usou-se a capacidade máxima do *hardware* (4 núcleos lógicos), desconfia-se que a paralelização via *OpenMP* não fez o mesmo, visto que usou-se o seu comando padrão `#pragma omp parallel for` sem a cláusula `num_threads(4)`.

Os autores acreditam que fazendo esses ajustes, os resultados computacionais confirmarão a previsão teórica e evidenciarão que recursos poderosos, como a computação paralela, se mal utilizados, podem perder boa parte de sua eficácia.

5. CONCLUSÃO

Com base no que foi apresentado, viu-se que a computação sequencial ainda desempenha um importante papel na solução de problemas computacionais, executando tarefas de forma sequencial, lembrando a lógica de funcionamento de uma fila indiana. Já a computação paralela, usando programas que fazem uso de múltiplos núcleos do processador (sistema paralelo), consegue resolver problemas mais rapidamente do que a programação tradicional.

Submeteu-se ambos os tipos de computação à solução do clássico problema de ordenação de vetores, onde eles tinham tamanhos que variavam de 15000 a 20000 elementos, sendo estes compostos de números inteiros aleatórios. O algoritmo dividiu o elementos dos vetores em blocos de quatro e cinco elementos, sendo ordenados, respectivamente, pelos algoritmos de ordenação *Insertion Sort* e *Selection Sort*. Posterior-

Table 1. Tempo médio de execução (em segundos) para 30 amostras por tamanho de vetor.

Tamanho do Vetor	Média Sequencial (s)	Média OpenMP (s)	Média Threads (s)
15000	0.002 373 566 7	0.012 530 833 3	0.002 382 400 0
16000	0.002 513 966 7	0.012 615 633 3	0.002 533 766 7
17000	0.002 709 000 0	0.012 589 966 7	0.002 718 433 3
18000	0.002 871 433 3	0.012 742 266 7	0.002 866 733 3
19000	0.003 052 566 7	0.012 954 000 0	0.002 997 700 0
20000	0.003 219 966 7	0.013 182 700 0	0.003 150 933 3

mente, os blocos já ordenados, eram unidos e eram subme-
tidos a uma última ordenação por meio do algoritmo *Merge*
Sort.

Empregando-se a paralelização na parte do algoritmo res-
ponsável pela ordenação, esperava-se que a paralelização de
alto nível (*OpenMP*) e a paralelização de baixo nível (*Pthreads*)
superassem o algoritmo sequencial, porém, este se mos-
trou superior ao *OpenMP* e igual em desempenho ao *Pthreads*,
evidenciando um mau uso da paralelização.

Os autores chegaram à conclusão de que, mesmo um re-
curso poderoso na computação, quando utilizado de forma
equivocada, pode perder boa parte de seus benefícios. Como
próxima etapa, os autores implementarão as correções ne-
cessárias e estão otimistas em obter resultados computacio-
nais que confirmem as previsões teóricas.

6. REFERENCES

- [1] Adam Drozdek, *Data structures and algorithms in C++*, Cengage Learning, 2012.
- [2] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein, "Introduction to algorithms (3-rd edition)," *MIT Press and McGraw-Hill*, 2009.
- [3] Peter Pacheco, *An introduction to parallel programming*, Elsevier, 2011.
- [4] Sirilak Ketchaya and Apisit Rattananurak, "Optimized n-deque partition dual-deque merge string sorting algorithm," *Scientific Reports*, vol. 16, no. 1, pp. 18436, 2026.
- [5] C Schepke, N Lucca, and J V F Lima, "Capítulo 1: Diretivas paralelas de openmp: um estudo de caso," file:///C:/Users/LENOVO/Downloads/ojs_{bc}, +CapAcessoem : 10deJulhode2026.
- [6] Tiago Araújo Neves, "Estrutura de dados e algoritmos," [https://sites.google.com/view/proftneves/in%C3%ADcio/disciplinas/estruturas-de-dados-e-algoritmos?](https://sites.google.com/view/proftneves/in%C3%ADcio/disciplinas/estruturas-de-dados-e-algoritmos?authuser=0) authuser=0, 2026, Acessado em: 10 de Julho de 2026.

Ana Fernanda Gomes Ascencio and Graziela Santos de Araújo, *Estruturas de dados: algoritmos, análise da complexidade e implementações em Java e C/C++*, Pearson, São Paulo, 1 edition, 2010.